

Adversarial FIDE Conformance Review and Regression Comparison of an Updated UCI Chess Engine

Manus AI

Independent Verification Report

September 2026

Updated main commit: 81a41818690afdf6f80b7ae3d822538dd795476f

Abstract—This paper independently reevaluates an updated main-branch snapshot of *Unchessed-UCI-Engine* without using the previous research branch, its PDF, or its added commits as engine inputs. The evaluation reviews the newly added FIDE scenarios, reruns the complete existing test surfaces, and compares results with a prior audit only after producing fresh evidence. The method combines FIDE Laws analysis, source inspection, canonical perft counts, an independently maintained `python-chess` oracle, a 1,000-position reachable-state sample, explicit move-list sequences, malformed-FEN probes, exact replay transcripts, and a seven-position verified mate corpus. Current main passes 123 default Rust tests, six ignored Rust tests, 385 Python tests, optimized Rust tests, all 28 recorded canonical perft checks, 7/7 synchronized mate cases, 1,004/1,004 non-terminal oracle-legal UCI moves, and 252/252 state-sequence cases. The prior dead-position failure is no longer reproduced: three tested dead-material classes now return `bestmove 0000`. However, the committed build script fails its own mate smoke gate because it sends `quit` after a fixed two-second sleep; a synchronized depth-six run returns the expected mate. Malformed FEN still produces an error message followed by a `bestmove`, so fail-closed protocol behavior is not established. The evidence supports improved, but bounded, FIDE-relevant conformance rather than complete compliance with all over-the-board Laws.

Index Terms—FIDE Laws of Chess, chess engine testing, adversarial verification, perft, differential testing, UCI, regression analysis.

I. INTRODUCTION

A chess engine’s evaluation strength does not establish correctness of legal move generation, state transitions, terminal adjudication, or engine–GUI protocol behavior. This distinction motivates an independent reevaluation of the updated *Unchessed-UCI-Engine* main branch after a project update that was not based on the prior research artifacts.

The precise current subject is remote `main` at commit 81a41818690afdf6f80b7ae3d822538dd795476f, titled “Close out the audit’s build/CI tail: linker preflight, release tests, CI.” The prior audited commit was 9c4f1f8e82dca8e4c729f275e8c15cb0e87e7b74. The updated source was cloned afresh, compiled, and executed independently. Historical results are included only as comparison data after the new runs completed.

The normative source is the official FIDE Laws of Chess taking effect 1 January 2023 [1]. The current FIDE Handbook additionally lists general regulations effective 1 March 2026

[2]; those are complementary administrative and technical rules, not a replacement for the Laws governing legal movement and game-state outcomes.

II. ADVERSARIAL SCOPE AND EVIDENCE MODEL

The adversarial verifier divided the FIDE matrix into engine-observable and over-the-board classes. Engine-observable scenarios include piece movement, blockers, pins, king safety, castling, en passant, promotion, checkmate, stalemate, dead positions, repetition-related state, halfmove boundaries, FEN validity, SAN where exposed, and UCI sequencing. Physical movement, touch-move, clocks, scoresheets, arbiter judgment, penalties, rapid/blitz administration, appeals, and tournament scoring were retained as explicit out-of-scope clauses rather than counted as passes.

The evidence ladder distinguishes source inspection (Level 3), completed automated tests (Level 4), and real-binary run-time execution (Level 5). A finite reachable sample cannot prove all chess positions. A legal selected move does not prove that every legal move was generated. Every numerical comparison in this paper is derived from raw logs or structured result files preserved on `manus/research-facilities`.

The main edge threats were illegal king-exposing moves, castling through attacked squares, stale en-passant rights, promotion errors, repetition-key errors, incorrect mate/halfmove precedence, dead-position omissions, malformed-input fallback, and premature protocol termination. The verifier also attacked the test harness itself: an initial streaming reader could misassociate results when preprocessing delayed a best-move, and a batch mate harness could race `quit` against the search. Those harness flaws were corrected before classifying the engine.

III. UPDATED SCENARIOS AND METHODS

A. FIDE Scenario Review

The new review covers all 54 rows in the saved scenario matrix. The engine-testable subset includes standard initialization; bishops, rooks, queens, knights, and pawns; captures and blockers; pins and check; promotion; en passant; both castling sides and castling rights; checkmate, stalemate, and dead material; FEN rejection; halfmove boundaries; and stateful

TABLE I
CURRENT MAIN EXISTING-TEST RESULTS

Surface	Result	Interpretation
Default Rust	123 pass; 0 fail; 6 ignored	Internal debug suite
Ignored Rust	6 pass; 0 fail	Deep/runtime checks
Python	385 pass; 22 skip	347 subtests passed
Release Rust	123 pass; 0 fail; 6 ignored	Optimized suite
Release ignored	6 pass; 0 fail	Optimized deep/runtime checks
Build script	Failed at smoke	Fixed sleep/quit race

move-list replay. For each applicable class, the adversarial design varied colors, edge geometry, occupied and empty destinations, legal and illegal controls, and state-reset conditions.

The FIDE 2023 Laws state that an immediately impossible-to-mate position is drawn under Articles 1.5 and 5.2.2, that checkmate completes the game, and that promotion is part of the same move [1]. They also distinguish the 50-move claim from the 75-move automatic draw and preserve checkmate precedence. The UCI source was treated as a protocol boundary: a result is trustworthy only when the caller waits for the corresponding `bestmove`.

B. Independent Differential Testing

The fresh UCI harness configured the updated binary with opening book and adaptive behavior disabled. It checked `uciok`, repeated `readyok`, 14 fixed FIDE-relevant positions, and 1,000 deterministic reachable positions generated from the initial position using seed 20260904 and up to 60 legal plies. Each non-terminal result was parsed as UCI and checked against `python-chess 1.11.2` legal moves. The terminal oracle included checkmate, stalemate, insufficient material, and automatic 75-move status.

A separate state-sequence harness generated 250 legal position `startpos moves ...` sequences and added explicit en-passant and castling sequences. Exact command streams and raw outputs for the explicit replay cases and every malformed-FEN probe were saved for inspection. The seven-position repository mate corpus was executed one isolated process at a time, waiting for `bestmove` before sending `quit`. A standalone Rust checker linked against the fresh main build evaluated all canonical perft positions and depths. All 252 sequences were then rerun at depths 1, 2, and 3 with one persistent UCI process, one thread, book disabled, synchronized result collection, and per-search wall-time/node capture.

IV. EXISTING TEST RESULTS

The exact committed build script was rerun. Its build and Rust test portions completed, but it exited at the forced-mate smoke assertion. The script sleeps for two seconds after `go depth 6` and then sends `quit`; the observed output was `bestmove alb1`, not the expected `ala8`. A corrected synchronized probe waited for completion and produced depth-one through depth-six mate information followed by `bestmove ala8`.

TABLE II
FRESH RUNTIME CONFORMANCE RESULTS

Test	Result	Edge coverage
Canonical perft	28/28	Start, Kiwipete, positions 3–6
UCI non-terminal moves	1,004/1,004	Fixed plus reachable legal states
Terminal responses	10/10	Mate, stalemate, insufficient material, 75-move oracle
Verified mate corpus	7/7	Both colors and multiple mate patterns
Move-list sequences	252/252	250 random plus EP/castling
Malformed FEN	5/5 errors	Each also emitted a bestmove

TABLE III
252-SEQUENCE MULTI-DEPTH BENCHMARK

Depth	Cases	Pass	Mean ms	P95 ms
1	252	252	2.889	8.610
2	252	252	3.665	11.775
3	252	252	6.682	16.730

The current Python suite completed in 21.54 seconds versus 22.33 seconds in the earlier capture. The current ignored debug suite completed in 52.76 seconds versus 54.39 seconds earlier. These timing differences are descriptive only: CPU scheduling, caches, and background load were not controlled, so they are not evidence of an engine-speed improvement.

V. FIDE-RELEVANT RESULTS

All 28 canonical perft checks passed: start position at depths 1–6, Kiwipete at depths 1–5, position 3 at depths 1–5, and positions 4–6 at depths 1–4. The most demanding counts matched exactly, including 119,060,324 start-position nodes at depth six and 193,690,690 Kiwipete nodes at depth five. These values originate from the established perft reference [3].

The fresh UCI differential test contained 1,014 valid positions. Of these, 1,004 were non-terminal and all 1,004 returned moves legal under the independent oracle. Ten positions were terminal under the expanded oracle, and all ten returned `bestmove 0000`. The seven-position verified mate corpus passed 7/7 when synchronized. The move-list/state suite passed 252/252. The multi-depth benchmark completed 756 synchronized searches, with 252 cases at each of depths 1–3 and 756/756 legal-or-terminal outputs. Mean wall time was 2.889, 3.665, and 6.682 ms, respectively; the corresponding 95th percentiles were 8.610, 11.775, and 16.730 ms. Mean node counts were 272.74, 456.82, and 985.82.

VI. OLD-VERSUS-NEW COMPARISON

Table IV compares the prior raw result files with the fresh main results. The denominators for the differential tests differ, so the percentages are not treated as a single longitudinal benchmark. The previous 1,011/1,011 legal-bestmove figure covered a 1,022-case run with six invalid inputs and five terminal cases excluded by the earlier harness classification. The fresh run uses a corrected terminal oracle and reports 1,004/1,004 non-terminal legal moves plus ten terminal responses.

TABLE IV
HISTORICAL COMPARISON, WITH SCOPE DIFFERENCES STATED

Metric	Earlier	Current	Difference
Rust default pass	123	123	Same; current main passes
Python pass	385	385	Same test count
Python time (s)	22.33	21.54	−0.79; uncontrolled
Ignored Rust time (s)	54.39	52.76	−1.63; uncontrolled
Nonterminal legal moves	1,011/1,011	1,004/1,004	Corrected denominator/oracle
Dead material	3/3 ordinary moves	3/3 0000	Prior failure no longer reproduced
Mate corpus runtime	Not independently recorded	7/7	New direct coverage
State sequences	Not run	252/252	New protocol coverage
Canonical perft	Passed prior branch	28/28 fresh main	No tested regression
Build smoke	Passed prior capture	Failed current script	Current gate race

VII. ADVERSARIAL FINDINGS

A. Dead Positions

The prior audit reported ordinary moves for king versus king, king and bishop versus king, and king and knight versus king. Current main returns 0000 for all three. Because the corrected oracle includes insufficient material, the earlier apparent current-main failures were recognized as a harness classification error rather than repeated engine failures. The updated result confirms improvement for these three constructions at Level 5, but it does not enumerate every possible dead position.

B. Smoke-Gate Synchronization

The current script’s fixed sleep/quit sequence is inconsistent with its own comment that a real GUI waits for `bestmove`. The exact two-second reproduction returns `alb1`; the synchronized depth-six run returns `ala8`. The engine behavior is therefore timing-sensitive under the script’s artificial termination, while the actual completed search passes the tested mate. The repository gate should read until `bestmove` with a bounded timeout.

C. Malformed FEN and Stale Output

The five malformed cases all produce parser-error text and all also emit a `bestmove`. This is not a process crash, but it is not fail-closed protocol behavior. A client that ignores informational output can attribute the `bestmove` to a rejected position. The back-rank pawn case remains a policy ambiguity because a promotion-ready FEN may be intentionally accepted for analysis, whereas missing kings, two same-color kings, invalid castling rights, and occupied en-passant state are clearly malformed under the tested parser contract.

D. Halfmove and Result Exposure

The search source preserves checkmate precedence before its halfmove draw guard, and the forced-mate test passes when synchronized. However, the UCI path does not expose a structured 50-move claim or 75-move result status to the client. FIDE Articles 9.3 and 9.6 distinguish those outcomes [1]. Move legality at high halfmove clocks was tested; complete result/API compliance remains unverified.

VIII. NOT VERIFIED AND THREATS TO VALIDITY

The reachable set is a deterministic finite sample, not the complete game graph. The perft corpus is selected, not exhaustive. The multi-depth benchmark uses depths 1–3 for protocol and regression measurement, not playing-strength evaluation. It does not establish all options, time controls, threads, neural-network configurations, memory sizes, or long-running stability. Old-versus-new wall-time differences are not controlled performance benchmarks; within-run depth timings are directly measured under a fixed one-thread configuration.

The audit cannot establish one-hand movement, touch-move, physical board restoration, clock pressing, flag fall, scoresheet rules, draw offers, arbiter intervention, penalties, appeals, venue/device conduct, rapid/blitz administration, or tournament scoring through a normal UCI interface. Current 2026 FIDE general regulations are relevant administrative context, but they do not make these physical procedures engine-testable [2].

IX. CONCLUSION

The updated main snapshot shows meaningful evidence of improved FIDE-relevant behavior. Canonical perft, optimized tests, the seven-position mate corpus, the 1,000-position reachable differential run, 252 state-sequence tests, and the 756-search multi-depth benchmark all pass within their declared scopes. The three prior dead-position failures are no longer reproduced. The principal current defects are the build script’s premature quit race and the parser’s post-rejection `bestmove` behavior. The strongest defensible conclusion is therefore **partial confirmation of tested move/state behavior, not complete FIDE compliance**. The exact source, raw outputs, comparison metrics, and this paper are committed on `manus/research-facilities`.

REFERENCES

- [1] FIDE, “FIDE Laws of Chess taking effect from 1 January 2023,” FIDE Handbook. [Online]. Available: <https://handbook.fide.com/chapter/e012023>
- [2] FIDE, “General Rules and Regulations effective from 1 March 2026,” FIDE Handbook. [Online]. Available: <https://handbook.fide.com/chapter/GeneralRulesAndRegulations032026>
- [3] Chess Programming Wiki, “Perft Results.” [Online]. Available: https://chessprogramming.org/Perft_Results

- [4] Shredder Chess, "Universal Chess Interface." [Online]. Available: <https://www.shredderchess.com/chess-features/uci-universal-chess-interface.html>
- [5] Niklas Fiekas, "python-chess Core documentation," version 1.11.2. [Online]. Available: <https://python-chess.readthedocs.io/en/latest/core.html>